Week 7: The Algorithm in the Courtroom

Naive Bayes

FIT1061: Introduction to Artificial Intelligence

2026-08-24

The Story: COMPAS and *Machine Bias*

In May 2016, a team of reporters at ProPublica published an investigation with a blunt title: **Machine Bias**. Its subject was a piece of software called **COMPAS** (short for Correctional Offender Management Profiling for Alternative Sanctions) sold by a company then called Northpointe. COMPAS takes a defendant's answers to a long questionnaire and returns a single **risk score from 1 to 10**: a number meant to capture how likely that person is to be arrested again. Across the United States, judges saw that number at **bail, sentencing, and parole** hearings. A score could influence whether someone went home to await trial or waited in a cell. And the formula behind it was **closed-source**: opaque to the defendant, to their attorney, and to the judge reading it.

Two defendants

The investigation opened with two people arrested in Broward County, Florida.

Vernon Prater was a 41-year-old white man picked up for shoplifting about $86 of tools from a hardware store. He had a serious record, including an earlier armed robbery. COMPAS scored him **3 out of 10: low risk**.

Brisha Borden was an 18-year-old Black woman arrested after she and a friend picked up a child's bicycle and scooter left in a driveway (worth roughly $80) and were spotted before they got far. She had no prior convictions. COMPAS scored her **8 out of 10: high risk**.

Then time passed. Prater, the "low-risk" man, went on to commit another crime and is now serving an eight-year prison term for a warehouse break-in. Borden, the "high-risk" teenager, was **not** arrested again. The score had it exactly backwards for both of them.

The finding, and the response

Two anecdotes prove nothing on their own. So ProPublica did what this week is really about: they **counted**. They linked around **7,000 Broward County COMPAS scores** from 2013–14 to what actually happened over the following two years. The pattern they found was that, among defendants who did **not** go on to reoffend, **Black defendants were roughly twice as likely as white defendants to have been falsely flagged as high-risk**.

Northpointe replied that its tool was **equally calibrated** across races: a given score meant the same reoffending rate whether the defendant was Black or white. Both claims were mathematically correct. They were measuring "fair" in two different ways, and, as we will hint at the end, those two definitions cannot both be satisfied at once when the underlying rates differ between groups.

This idea is nearly a century old

It is tempting to think of algorithmic risk scoring as a modern intrusion, but scoring a person by adding up background factors is not new. In **1928**, the sociologist **Ernest Burgess** built a scheme that scored parolees on background factors (a strong work record and a farming background pointing one way, a criminal-underworld background pointing the other) and proposed a **summary sheet** so a parole board could read the factors at a glance. COMPAS, released in 1998, is a direct descendant. Burgess added his factors up **by hand**. This week we build the modern version, where the factors are **words in a message** and the "adding up" becomes **multiplying probabilities**. We will build it on low-stakes spam first, then notice that the machine deciding your inbox is the **same kind of machine** being asked to score a defendant. **The structure is the same. The stakes are not.**

From One Word to Many

Last week you learned to flip a single conditional. Given the one word **"free"**, Bayes' rule turned the likelihood you can count, $P(\text{"free"} \mid \text{spam})$, into the posterior you actually want:

$$P(\text{spam} \mid \text{"free"}) = \frac{P(\text{"free"} \mid \text{spam})\, P(\text{spam})}{P(\text{"free"})}.$$

That worked beautifully for **one** word of evidence. But a real message is not one word; it is a whole **bag of words**: *"free", "prize", "call", "now"*, and so on. To do the exact calculation we would need the joint likelihood of every word appearing together:

$$P(\text{free, prize, call, now} \mid \text{spam}).$$

Why the exact answer is impossible

To count that joint directly, we would need a training example for **every possible combination** of words being present or absent. For a modest vocabulary of just 1,000 words, that is 2^{1000} combinations: a number larger than the count of atoms in the observable universe. We could never gather enough labelled email to fill in a table that size. The exact joint is, for all practical purposes, **impossible to count**.

The naive assumption

The way out is a deliberate, slightly reckless simplification.

> **i** Definition: the Naive Bayes assumption
>
> Assume the words are **independent given the class**. Then the joint likelihood collapses into one probability per word, multiplied together:
>
> $$P(w_1, \dots, w_n \mid c) \approx \prod_i P(w_i \mid c).$$

Instead of one gigantic table, we now need only a **small probability for each word on its own** (how often "free" shows up in spam, how often "meeting" shows up in ham) and we multiply them.

The assumption is called *naive* because it is **false**: "free" and "prize" clearly travel together in spam; they are not independent. But we do not actually need the probabilities to be correct; we only need the **ranking** to come out right. We are going to compare a spam score against a ham score and pick the larger. The naive assumption distorts the exact numbers (it tends to make them overconfident), yet it usually preserves the **order** of the two scores, which is all the decision depends on. As the statistician George Box put it, *"all models are wrong, but some are useful."*

Naive Bayes

We can now state the whole method. For each class (here spam or ham) we compute a **score**, and we predict whichever class scores highest. Nothing limits this to two classes: a news app sorting stories into ten topics computes ten scores and picks the largest, using exactly the same rule. Two classes is just the simplest case.

Definition: the Naive Bayes scoring rule

For each class c,

$$\text{score}(c) = P(c) \prod_i P(w_i \mid c),$$

where the product runs over the words w_i that appear in the message. **Predict the class with the highest score.**

Three things are worth unpacking:

- $P(c)$ is the **prior**: how common the class is before we read any words at all. If half our training email is spam, $P(\text{spam}) = 0.5$.
- We would, strictly, divide each score by $P(w_1, \ldots, w_n)$, the overall chance of seeing this exact message. But that denominator is the **same for every class**, so it does not change which score is largest. We drop it.
- We multiply likelihoods only for the words that **actually appear** in the message.
- If the two scores tie, we predict **ham** (the harmless class) so we never flag a message spam without positive reason.

Training is just counting

Here is the part that surprises people. There is **no optimisation, no gradient descent, no iterative tuning** in training a Naive Bayes classifier. To learn the model you simply **count**:

- **Priors:** for each class, count how many training messages belong to it and divide by the total number of messages.
- **Likelihoods:** for each word and each class, count how many messages *in that class* contain the word, and divide by the number of messages in that class.

That is the entire training step. The likelihood of a word is just the **fraction of that class's messages that contain it**: a document-presence count, not a tally of how many times the word repeats. To make the procedure explicit, in language-neutral pseudocode:

```
COMPUTE_PRIORS:
    for each class c:
        prior[c] <- (messages in c) / (total messages)

COMPUTE_LIKELIHOODS:
    for each class c, each word w in the vocabulary:
        likelihood[c][w] <- (messages in c containing w) / (messages in c)

PREDICT(message):
```

```
for each class c:
    score[c] <- prior[c]
    for each distinct word w present in the message and in the vocabulary:
        score[c] <- score[c] * likelihood[c][w]
return the class c with the largest score   (tie -> ham)
```

Worked Example: "free meeting"

Nothing makes the method concrete like running it by hand. Suppose our entire training set is just **six emails**: three spam and three ham:

#	label	words
1	**spam**	free, prize, winner
2	**spam**	free, offer, click
3	**spam**	winner, call, now
4	ham	meeting, agenda, tuesday
5	ham	project, update, meeting
6	ham	free, lunch, friday

A new message arrives: **"free meeting"**. Spam or ham?

Step 1: the priors

Three of the six emails are spam and three are ham, so

$$P(\text{spam}) = P(\text{ham}) = \tfrac{3}{6} = 0.5.$$

Step 2: the likelihoods

We need each word of the test message, in each class. Count the messages containing the word and divide by the class size (3):

- $P(\text{free} \mid \text{spam}) = \tfrac{2}{3}$: "free" appears in spam emails 1 and 2.
- $P(\text{free} \mid \text{ham}) = \tfrac{1}{3}$: "free" appears in ham email 6.
- $P(\text{meeting} \mid \text{spam}) = \tfrac{0}{3}$: "meeting" appears in **no** spam email.
- $P(\text{meeting} \mid \text{ham}) = \tfrac{2}{3}$: "meeting" appears in ham emails 4 and 5.

Step 3: the scores

Multiply the prior by the likelihoods for the words in the message:

$$\text{score}_{\text{spam}} = 0.5 \cdot \tfrac{2}{3} \cdot \mathbf{0} = \mathbf{0},$$
$$\text{score}_{\text{ham}} = 0.5 \cdot \tfrac{1}{3} \cdot \tfrac{2}{3} = \tfrac{1}{9}.$$

Since $\frac{1}{9} > 0$, the classifier predicts **ham**.

The trouble hiding in that zero

Look at what just happened. The word **"free"** is a strong spam signal (it is more common in spam than in ham) yet the message was classified ham anyway. The culprit is $P(\text{meeting} \mid \text{spam}) = 0$. Because "meeting" never appeared in a spam email in our tiny training set, its likelihood is exactly zero, and **a single zero factor drags the whole product to zero**, no matter how much other evidence points to spam.

> **!** Key idea: the zero-frequency problem
>
> The score is a **product**, and a product cannot recover from a single zero (a sum could). One word never seen in a class sets that class's score to zero, overruling every other word. This is a **bug, not a feature**; and with a vocabulary of thousands of words, some unseen word will appear in almost every new message.

The Zero-Frequency Fix: Laplace Smoothing

The fix is as old as the problem is deep, and it has a name: **Laplace smoothing**, after Pierre-Simon Laplace (1749–1827). The idea is to **pretend we saw a little of everything**: to give every word one small imaginary observation, so that no likelihood is ever exactly zero.

> **i** Definition: Laplace smoothing (binary form)
>
> $$P(w \mid c) = \frac{\text{count}(w, c) + 1}{\text{count}(c) + 2},$$
>
> where $\text{count}(w, c)$ is the number of messages in class c containing word w, and $\text{count}(c)$ is the number of messages in class c.

The two adjustments each have a plain reading. The $+1$ in the numerator is the single imaginary observation added for every word, so nothing is ever exactly zero. The $+2$ in the denominator

reflects the two outcomes a word feature can take: **present** or **absent** in a message. (This is the binary, document-presence form of smoothing, which matches how we count likelihoods here.)

Re-running the verdict

Let us smooth the "free meeting" example and try again. With $\text{count}(c) = 3$ for each class:

- $P(\text{meeting} \mid \text{spam}) = \frac{0+1}{3+2} = \frac{1}{5}$: **no longer zero.**
- $P(\text{free} \mid \text{spam}) = \frac{2+1}{3+2} = \frac{3}{5}$.
- $P(\text{free} \mid \text{ham}) = \frac{1+1}{3+2} = \frac{2}{5}$.
- $P(\text{meeting} \mid \text{ham}) = \frac{2+1}{3+2} = \frac{3}{5}$.

The scores become

$$\text{score}_{\text{spam}} = 0.5 \cdot \tfrac{3}{5} \cdot \tfrac{1}{5} = 0.06,$$
$$\text{score}_{\text{ham}} = 0.5 \cdot \tfrac{2}{5} \cdot \tfrac{3}{5} = 0.12.$$

The prediction is **still ham**, but notice that spam is now a genuine possibility rather than a mathematical impossibility. **Smoothing does not flip the answer; it stops the catastrophic zero** from silencing every other word.

Choosing which words to count

A companion idea is **feature selection**: not every word earns its place in the vocabulary. Words like *"the"*, *"a"*, and *"is"* (the **stop-words**) appear about equally in spam and ham, so they carry no signal and only add noise. We can drop them and keep the **most informative** words: those that are far more common in one class than the other. A practical measure of informativeness is how far apart a word's two likelihoods are, $|P(w \mid \text{spam}) - P(w \mid \text{ham})|$. A smaller, well-chosen vocabulary often **beats** the full one, because it carries fewer noisy near-zeros and is less prone to memorising quirks of the training set.

Measuring the Classifier

Building a classifier is only half the job; we have to know **how good it is**. Counting the raw fraction it gets right (its **accuracy**) turns out to hide the very thing we care about, so we lay the results out in a **confusion matrix**: a 2×2 tally of what the classifier predicted against what was actually true. With spam as the "positive" class:

	predicted spam	predicted ham
actual spam	**TP** (spam caught)	**FN** (spam missed)
actual ham	**FP** (ham wrongly flagged)	**TN** (ham correctly kept)

The four cells have names: **true positives** (TP), **false negatives** (FN), **false positives** (FP), and **true negatives** (TN). From these four counts we build three headline numbers.

i Definition: precision, recall, accuracy

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad \text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{total}}.$$

In words: **precision** asks *of the messages we flagged as spam, how many really were spam?*: it measures how much we can trust a positive verdict. **Recall** asks *of all the real spam, how much did we actually catch?*: it measures how much slips through. (These are the plain-language names for what statisticians call Type-I and Type-II error, but the plain names are clearer.)

A worked report card

Suppose we run our classifier on **eight held-out messages** it has never seen (three genuine spam and five genuine ham) and it produces this confusion matrix:

	predicted spam	predicted ham
actual spam	$\text{TP} = 2$	$\text{FN} = 1$
actual ham	$\text{FP} = 0$	$\text{TN} = 5$

Reading the three numbers off the cells:

$$\text{precision} = \frac{2}{2+0} = 100\%, \qquad \text{recall} = \frac{2}{2+1} = 67\%, \qquad \text{accuracy} = \frac{2+5}{8} = 88\%.$$

The single mistake was a message like *"free msg claim your reward call the number below"*: a real spam that the classifier waved through as ham. That is a **false negative**: spam that slipped past. Notice the story the numbers tell together: **precision 100%** means the filter never once cried wolf (every message it flagged really was spam), but **recall 67%** means it let one real spam through.

! Key idea: accuracy alone is a trap

A lazy classifier that simply labelled **every** message ham would score $\frac{5}{8} = 62.5\%$ accuracy here, without catching a single spam. Accuracy hides which kind of error the classifier makes. Precision and recall pull those errors apart, and which one you care about depends on what is at stake.

Back to the Courtroom

Now watch the same machinery change meaning. The spam filter and COMPAS are, structurally, the **same kind of classifier**, but the cost of each error is worlds apart:

	Spam classifier	COMPAS
input	words in a message	age, prior arrests, employment…
output	spam / ham	high-risk / low-risk
false positive	a real email sent to junk (*annoying*)	flagged high-risk and **detained** — yet never re-arrested
false negative	spam reaches the inbox (*annoying*)	someone later re-arrested is released
stakes	convenience	**liberty**

For a spam filter, we argued a false positive (a real email dumped in a junk folder you may never open) is worse than a false negative, so we would tune for **high precision**. But flip to COMPAS and the false positive is no longer a lost email; it is **a person held in a cell who should have gone home**. The same tuning question now decides someone's freedom, and which error is worse depends entirely on what is at risk.

There is a second lesson hiding in the COMPAS story. The tool weighs **137 questionnaire items** behind a closed formula, yet it is only about **65% accurate** — against about **55%** for a lazy rule that simply calls everyone "low-risk" (Broward's two-year re-arrest rate is roughly 45%). Researchers have shown that a rule you can read aloud in one breath (roughly, *"more than three prior offences, or an 18–20-year-old male, or 21–23 with two or more priors → predict re-arrest; otherwise not"*) matches that accuracy (Dressel & Farid, 2018). If a two- or three-factor rule does as well as the black box, it is fair to ask why the tool actually deciding bail is the one **no one is allowed to inspect**.

More than one definition of "fair"

Return, finally, to the collision at the heart of *Machine Bias*. Northpointe said the tool was *"equally calibrated"* across races (a given score meant the same reoffending rate for everyone). ProPublica said it had *"unequal false-positive rates across groups."* **Both were telling the truth.** They had simply chosen different definitions of fairness, and it turns out those definitions **cannot all hold at once** whenever the base rates differ between groups. Exactly *why* that is unavoidable, and what it means for building systems that touch people's lives, is something we make precise later.

Summary

Key takeaways:

- **The exact joint is impossible; the naive assumption rescues it.** Treating words as independent given the class turns an uncountable 2^n-way joint into one probability per word, multiplied.
- **Naive Bayes scores each class and picks the winner:** $\text{score}(c) = P(c) \prod_i P(w_i \mid c)$; the shared denominator cancels.
- **Training is just counting.** Priors are class fractions; likelihoods are the fraction of a class's messages containing each word. No optimisation.
- **A single zero collapses a score.** One word unseen in a class zeros its whole product, fixed by **Laplace smoothing** in its binary form, $(\text{count} + 1)/(\text{count} + 2)$.
- **Accuracy hides the error you care about.** The **confusion matrix** splits mistakes into false positives and false negatives; **precision** and **recall** measure the two, and which one matters depends on the stakes.
- **Same machine, opposite stakes.** The filter that sorts your inbox is the machine being asked to score a defendant; but there a false positive is a person in a cell.

Key vocabulary:

Term	Meaning
Naive Bayes assumption	Treat words as independent given the class, so the joint likelihood becomes a product of per-word likelihoods
Prior $P(c)$	How common a class is before reading any words; its fraction of the training set
Likelihood $P(w \mid c)$	The fraction of class c's messages that contain word w
Score	$P(c) \prod_i P(w_i \mid c)$; predict the class with the largest score
Zero-frequency problem	One unseen word gives likelihood 0, collapsing the whole product to 0

Term	Meaning
Laplace smoothing	The zero fix: $(\text{count} + 1)/(\text{count} + 2)$; one imaginary observation per word
Confusion matrix	A 2×2 tally of predictions against truth: TP, FP, FN, TN
Precision	$\text{TP}/(\text{TP} + \text{FP})$: of what we flagged, how much was right
Recall	$\text{TP}/(\text{TP} + \text{FN})$: of the real positives, how much we caught
False positive	A "yes" when the truth is "no": a binned email, or someone detained who was never re-arrested

What's next: this week the machine only ever **counted**; it never adjusted a single number. Next we meet a machine that **tunes its own numbers** to fit the data: the first step toward learning.

In the Lab

This week: build a many-word classifier from a handful of labelled emails, then measure how well it works.

You will put every idea above into working code, starting from a small set of labelled messages.

1. **Build the classifier.** Count the priors (class fractions) and the likelihoods (per-word, per-class presence fractions), then score a new message by multiplying the prior by the likelihoods of the words it contains, and predict the higher score.
2. **Watch a zero collapse a score.** Feed in a message containing a word never seen in one class, and see the whole product drop to zero even when other evidence points the other way: the zero-frequency problem in action.
3. **Measure the classifier.** Run it on a held-out set, tally the results into a confusion matrix, and compute **precision**, **recall**, and **accuracy**, then decide which error would matter more if these were people, not emails.

Everything you need is counting and division. There is no new mathematics beyond what is in these lecture notes; only careful bookkeeping.